

Last updated on **Apr 29, 2026**

Access and manage saved queries

This article outlines how to use the Databricks UI to access and manage queries.

View queries

You can view queries using the following methods:

- Click  **Workspace** in the sidebar. Queries are viewable, by default, in the **Home** folder. Users can organize queries into folders in the workspace browser along with other Databricks objects.
- Click  **Queries** in the sidebar. Objects in the **Queries** windows are sorted in reverse chronological order by default. You can reorder the list by clicking the **Created at** column heading. Type into the **Filter queries** text box to filter by Name, Tag, or Owner.

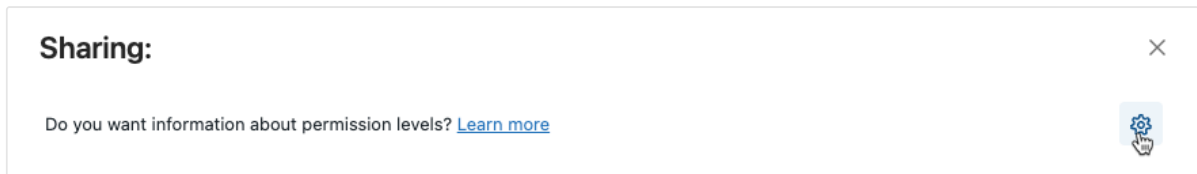
Organize queries into folders in the workspace browser

You can organize queries into folders in the [workspace browser](#) and other Databricks objects. See [Workspace browser](#).

Transfer ownership of a query

You must be a workspace admin to transfer ownership of a query. Service principals and groups cannot be assigned ownership of a query. You can also transfer ownership using the [Permissions API](#).

1. As a workspace admin, log in to your Databricks workspace.
2. In the sidebar, click **Queries**.
3. Click a query.
4. Click the **Share** button at the top right to open the **Sharing** dialog.
5. Click on the gear icon at the top right and click **Assign new owner**.



6. Select the user to assign ownership to.
7. Click **Confirm**.

Configure query permissions

Workspace admins and the query creator are automatically granted permissions to control which users can manage and run queries. You must have at least CAN MANAGE permission on a query to share queries.

Queries support two types of sharing settings:

- **Run as viewer:** The viewer's credential is used to run the query. The viewer must also have at least CAN USE permissions on the warehouse. In the legacy SQL editor, users can only be granted the CAN EDIT permission when the sharing setting is set to **Run as viewer**.
- **Run as owner:** In the new SQL editor, the owner's credential is used for legacy alerts and Lakeflow Jobs. The viewer's credential is always used to run the query. In the legacy SQL editor, the owner's credential is used to run the query.

For more information on query permission levels, see [Query ACLs](#).

1. In the sidebar, click **Queries**.
2. Click a query.

3. Click the **Share** button at the top right to open the **Sharing** dialog.

The screenshot shows the 'Sharing: Retail Revenue & Supply Chain' dialog. At the top, there's a title bar with a close button. Below it, a link to 'Learn more' about permission levels is visible. A search bar prompts the user to 'Type to add multiple users or groups'. Below the search bar, two users are listed: a user with an email ending in '@databricks.com' with 'Can Manage (inherited)' permissions, and 'Admins' with 'Can Run (inherited)' permissions. A section titled 'Sharing settings' contains a 'Credentials' dropdown menu set to 'Run as owner (owner's credential)'. Below this is an orange warning box titled 'Permission adjustment' stating that when 'Run as owner' is selected, all viewers will see the latest results, and users with 'Can Manage' or 'Can Edit' will be downgraded to 'Can Run'. At the bottom left, there is a 'Copy link' button.

4. Follow the steps based on the permission type you want to grant:

5. Search for and select the groups and users, and assign the permission level.

6. Click **Add**.

7. In the **Sharing settings > Credentials** field at the bottom, select either **Run as viewer** or **Run as owner**.

You can also copy the link to the query in the Sharing dialog.

Admin access to all queries

A Databricks workspace admin user has view access to all queries in the workspace. When the **All Queries** tab is selected, a workspace admin can view and delete any queries. However, a workspace admin can't edit a query when sharing setting credentials are set to **Run as owner**.

To view all queries:

1. Click  **Queries** in the sidebar.
2. Click the **All queries** tab near the top of the screen.

Creating queries in other environments

You can create queries without using the Databricks UI using the Rest API, a JDBC/ODBC connector, or a partner tool.

See [SQL connectors, libraries, drivers, APIs, and tools](#) to run SQL commands and browse database objects in Databricks.

You can also create a query with the [Databricks Terraform provider](#) and [databricks_sql_query](#).

See [Technology partners](#) to learn about partner tools you can use through Partner Connect.

Is this page

as this page helpful?

☐ Yes

☐ No

Last updated on **Apr 29, 2026**

Query history

NOTE

If your workspace is enabled for serverless compute, your query history also contains all SQL and Python queries run on serverless compute for notebooks and jobs. See [Connect to serverless compute](#).

You can use the information available through this screen to help you debug issues with queries.

This section describes how to work with query history using the UI. To work with query history using the API, see [Query History](#).

View query history

You can view previous query runs if you are the query owner. Other users can view query runs if they have at least CAN VIEW access to the SQL warehouse that executed the query.

To view previous runs for a query:

1. Click  **Query History** in the sidebar.
2. Optionally, use the filters near the top of the page to filter the list by user, date range, compute, duration, query status, statement type, statement ID, and [query tags](#).

NOTE

Databricks SQL queries shared by a user with **Run as Owner** permissions to another user with CAN RUN permissions appear in the query history of the user executing the query and not the user that shared the query.

 Ask Genie

View query details

To view details about a query, such as the complete query statement, query source, duration, number of rows returned, and I/O performance, or [execution details](#).

1. Access the query history as outlined in the previous section.
2. Click the query text you want to view. A summary panel opens to the right.

✓ Finished
New Query Profile: ON ▾
⋮
✕

Serverless compute

ID: e802a963-ab26-428a-b726-3d7ebeaaaaa27

SELECT
o.o_orderkey,
...10 more lines

Bytes read
174.26 MB
▽ 79%

Bytes written
0

Rows read
8

Rows written
0

Files read
4
▽ 64%

Files written
0

See query profile >

See longest operations for this query

Query wall-clock duration ⓘ

Total wall-clock duration		506 ms
Optimizing query & pruning files ⓘ	46%	235 ms
Executing ⓘ	54%	271 ms

Start time

3/20/2025, 03:43:53.885 PM

End time

3/20/2025, 03:43:54.391 PM

Result fetching by client ⓘ

62 ms

Query Source

> New Query Profile Demo / Cell(ID: 732...88187857298)

Aggregated task time ⓘ

Tasks total time

109 ms

Tasks time in Photon

71 %

See query profile

- **Query status:** The query is tagged with its current status: **Queued**, **Running**, **Finished**, **Failed**, or **Cancelled**.

- **User and compute details:** See the username, compute type, and runtime details for this query run.
- **ID:** This is the universally unique identifier (UUID) associated with the given query execution.
- **Query statement:** This section includes the complete query statement. If the query is too long to be shown in the preview, click ... **more lines** to view the full text.
- **Query metrics:** Popular metrics for query analysis are shown under the query text. The filter icons that appear with some metrics indicate the percent of data pruned during scanning.
- **See query profile:** A preview the query profile directed acyclic graph (DAG) is shown in this summary. This can be helpful for quickly estimating query complexity and execution flow. Click **See query profile** to open the detailed DAG.
- **See longest operators for this query:** Click this button to open the **Top operators** panel. This panel shows the longest running operators in the query.
- **Query wall-clock duration:** The total elapsed time between the start of scheduling and the end of the query execution is provided as a summary. A detailed breakdown of scheduling, query optimization and file pruning, and execution time appears below the summary.
- **Query source:** Click on the name of the listed object to go to the query source.
- **Aggregated task time:** View the combined time it took to execute the query across all cores of all nodes. It can be significantly longer than the wall-clock duration if multiple tasks are executed in parallel. It can be shorter than the wall-clock duration if tasks waited for available nodes.
- **Input/Output (IO):** View details about the data read and written during the query execution.

3. For more detailed information about the query's performance, including its execution plan, click **View Query Profile** near the bottom of the page. For more details, see [Query profile](#).

Terminate an executing query



To terminate a long-running query started by you or another user:

1. Access the query history as outlined in the previous section.
2. Click the query text.
3. Next to **Status**, click **Cancel**.

NOTE

Cancel only appears when a query is running.

The query is terminated, and its status changes to **Canceled**.

NOTE

Statements that use Lakeflow Spark Declarative Pipelines compute can only be canceled from the Pipelines UI.

Access the query history system table

Databricks system tables include a query history table (`system.query.history`) where privileged users can view tabular query history records from across your account. By default, only admins have access to your account's system tables. For information on enabling and accessing the query history system table, see [Query history system table reference](#).

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **May 11, 2026**

Query tags

⚠️ PREVIEW

This feature is in [Public Preview](#).

This page describes how to use query tags to group, filter, and attribute costs for SQL workloads on Databricks SQL warehouses.

Query tags are custom key-value pairs (for example, `team:marketing` or `dbt_model_name:some_model_name`) that you apply to SQL workloads. These tags appear in the `system.query.history` table and on the **Query History** page of the Databricks UI, and are returned in responses to the [ListQueries API](#) if non-empty. Query tags allow you to group queries by business context, attribute warehouse costs, and identify sources of long-running queries.

⚠️ WARNING

Tag data is stored as plain text and might be replicated globally. Don't use tag keys or values that contain passwords, personal information, or other sensitive data.

Requirements

Before using query tags, verify the following:

- Confirm you can access the `system.query.history` table, specifically the `query_tags` column. If not, contact your account admin. See [Query history system table reference](#).
- Minimum version requirements based on where you set query tags:
 - **dbt:** dbt-databricks 1.11.0

- **Power BI:** October 2025 release
- **Python Connector:** v4.1.3 (session-level), v4.2.6 (statement-level)
- **Node.js Connector:** v1.12.0
- **Go Connector:** v1.9.0
- **JDBC driver (OSS):** v3.0.3

Query tag scope

Query tags are scoped to [Databricks SQL sessions](#). You can set them when creating a session as a configuration parameter, or within a session using the `SET QUERY_TAGS` SQL statement. After you set tags, all subsequent statements in the session are associated with those tags.

Set query tags

You can set query tags using session configuration parameters or SQL statements.

Use session configuration parameters

Set query tags using the `query_tags` configuration parameter (or `ssp_query_tags` in some drivers) when creating a session. The value is a serialized set of key-value pairs, with a colon (:) separating keys and values, and a comma (,) separating pairs. This string format is accepted by the client interfaces listed in [Set query tags from tools and interfaces](#). For instructions on configuring sessions in your specific client, see the examples below.

If a colon (:), comma (,), or backslash (\) appears in a value, escape it with a leading backslash (\):, \\, or \\\\. Backslash (\) must be escaped in both keys and values.

The following example specifies tags `team=eng`, `cost_center=701`, a key-only tag `exp`, and a `metadata` tag with a JSON blob:

Text

 Ask Genie

```
query_tags = team:eng,cost_center:701,exp,metadata:
{"foo"\:"bar"\,"baz"\:1}
```

Use SQL statements

Use the `SET QUERY_TAGS` statement to set, read, or remove query tags for the current session.

For syntax, parameters, and examples, see [SET QUERY_TAGS](#).

Set query tags from tools and interfaces

Databricks UI

Use the `SET QUERY_TAGS` SQL statement anywhere you can submit SQL to a warehouse, including the SQL editor, notebooks, and dashboards. See [SET QUERY_TAGS](#).

Databricks SQL Statement Execution API (SEA)

Include the `query_tags` field in the request body of [POST /api/2.0/sql/statements](#) to apply statement-level tags. Tags set this way apply only to that specific statement execution.

Bash

```
curl -X POST "https://${DATABRICKS_HOST}/api/2.0/sql/statements" \
-H "Authorization: Bearer ${DATABRICKS_TOKEN}" \
-H "Content-Type: application/json" \
-d '{
  "warehouse_id": "abc123",
  "statement": "SELECT * FROM samples.nyctaxi.trips LIMIT 10",
  "query_tags": [
    { "key": "team", "value": "engineering" },
    { "key": "env", "value": "prod" }
```

 Ask Genie

```
}'
```

dbt

Minimum version: dbt-databricks 1.11.0

The following reserved query tag keys are set automatically for all dbt runs and can't be overridden:

JSON

```
{
  "dbt_model_name": "my_model",
  "dbt_core_version": "1.10.7",
  "dbt_databricks_version": "1.11.0",
  "dbt_materialized": "incremental"
}
```

NOTE

Default tags are subject to syntax limitations. Model names that include unescaped colons or commas result in `tag_invalid: true` in the system table.

You can add custom query tags at both project and model levels. Model configuration takes priority over connection configuration.

Project-level tag (`~/.dbt/profiles.yml`):

YAML

```
your_profile_name:
  target: dev
  outputs:
    dev:
      query_tags: '{"team": "marketing", "cost_center": "3000"}'
```

Model-level tag (`~/.dbt/dbt_project.yml`):

YAML

```
name: 'your_project'
version: '1.0.0'
config-version: 2
models:
  your_model:
    +query_tags: '{"team": "content-marketing"}'
```

Result with both configurations:

JSON

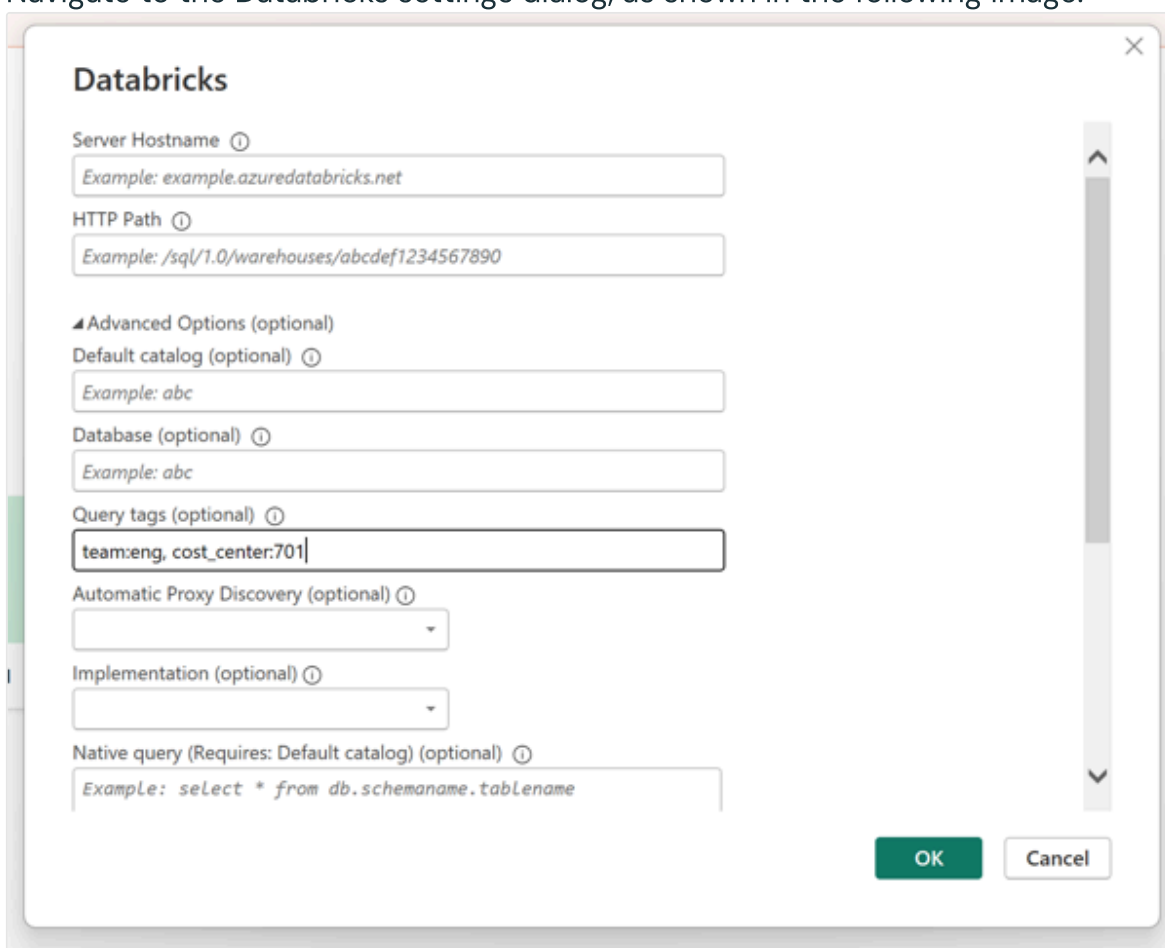
```
{
  "team": "content-marketing",
  "cost_center": "3000",
  "dbt_model_name": "model.dev.your_model",
  "dbt_core_version": "1.10.7",
  "dbt_databricks_version": "1.11.0",
  "dbt_materialized": "incremental"
}
```

Power BI

Minimum version: October 2025 release

1. Configure the connection to the warehouse.

2. Navigate to the Databricks settings dialog, as shown in the following image.



The screenshot shows a 'Databricks' settings dialog box with a close button (X) in the top right corner. The dialog contains several input fields and a section for advanced options. The fields are: 'Server Hostname' with an example 'example.azuredatabricks.net'; 'HTTP Path' with an example '/sql/1.0/warehouses/abcdef1234567890'; 'Advanced Options (optional)' section containing 'Default catalog (optional)' with example 'abc', 'Database (optional)' with example 'abc', 'Query tags (optional)' with 'team:eng, cost_center:701', 'Automatic Proxy Discovery (optional)' with a dropdown arrow, 'Implementation (optional)' with a dropdown arrow, and 'Native query (Requires: Default catalog) (optional)' with example 'select * from db.schemaname.tablename'. At the bottom right are 'OK' and 'Cancel' buttons.

3. In the **Query tags** text box, enter query tags using the [session configuration parameter syntax](#).

4. Click **OK**.

NOTE

Changing query tags and clicking **OK** starts a new session. Previously set tags are discarded.

Enable auto query tags

Auto query tags automatically attach Power BI context metadata to queries sent to Databricks warehouses without requiring you to manually specify tag values. To enable auto query tags, modify the M query in the Power Query Editor to include `EnableAutoQueryTags="true"` in the connection parameters.

1. In Power BI Desktop or the Power BI service, find the semantic model that includes the query you want to tag.

2. Open the **Power Query Editor**.
3. Open the **Advanced Editor**.
4. Add `EnableAutoQueryTags="true"` to the options in the `DatabricksMultiCloud.Catalogs` call.

The following example shows an M query with auto query tags enabled:

Powerquery

```
let
    Source = DatabricksMultiCloud.Catalogs(
        "myworkspace.cloud.databricks.com",
        "/sql/1.0/warehouses/abc123",
        [Catalog=null, Database=null, EnableAutoQueryTags="true",
        EnableAutomaticProxyDiscovery=null, Implementation="2.0"]),
    samples_Database = Source{[Name="samples",Kind="Database"]}[Data],
    nyctaxi_Schema = samples_Database{[Name="nyctaxi",Kind="Schema"]}[Data],
    trips_Table = nyctaxi_Schema{[Name="trips",Kind="Table"]}[Data]
in
    trips_Table
```

When auto query tags are enabled, Power BI automatically attaches the following reserved tags (prefixed with @@) to queries. The tags available depend on the Power BI connection mode:

Tag key	DirectQuery	Import
@@powerbi_activity_id	Yes	Yes
@@powerbi_dataset_id	Yes	No
@@powerbi_report_id	Yes	No
@@powerbi_visual_id	Yes	No

NOTE

 Ask Genie

Auto query tags are not attached to metadata queries (such as catalog or schema discovery queries) in any mode.

Parameterize the setting

If your semantic model contains multiple tables, Databricks recommends defining `EnableAutoQueryTags` as a Power Query parameter. This lets you enable or disable auto query tags across all queries from a single location instead of editing each query individually.

1. In the **Power Query Editor**, select **Manage Parameters** > **New Parameter**.
2. Create a parameter named `EnableAutoQueryTags` with a type of **Text** and a current value of `"true"`.
3. In each query, replace the hardcoded `EnableAutoQueryTags="true"` with a reference to the parameter.

The following example shows an M query that references the parameter:

Powerquery

```
let
    Source = DatabricksMultiCloud.Catalogs(
        "myworkspace.cloud.databricks.com",
        "/sql/1.0/warehouses/abc123",
        [Catalog=null, Database=null,
        EnableAutoQueryTags=EnableAutoQueryTags,
        EnableAutomaticProxyDiscovery=null, Implementation="2.0"]),
    samples_Database = Source{[Name="samples",Kind="Database"]}[Data],
    nyctaxi_Schema = samples_Database{[Name="nyctaxi",Kind="Schema"]}[Data],
    trips_Table = nyctaxi_Schema{[Name="trips",Kind="Table"]}[Data]
in
    trips_Table
```

With this approach, you can toggle auto query tags for all queries in your semantic model by updating the parameter value once, without editing individual queries.

Tableau

1. Configure the connection to the warehouse.

2. Navigate to the **Initial SQL** tab.
3. Enter query tags using the [SET QUERY_TAGS](#). You can include Tableau parameters in the key or value.
4. Click **Sign In** to save and authenticate.

NOTE

Changing the Initial SQL and clicking **Sign In** starts a new session. Previously set tags are discarded.

Python Connector

Minimum version: v4.1.3 (session-level), v4.2.6 (statement-level)

Session-level tags:

Python

```
from databricks import sql
import os

with sql.connect(
    server_hostname = os.getenv("DATABRICKS_SERVER_HOSTNAME"),
    http_path       = os.getenv("DATABRICKS_HTTP_PATH"),
    access_token    = os.getenv("DATABRICKS_TOKEN"),
    query_tags = {"team": "engineering", "dashboard": "abc123"}
) as connection:
    with connection.cursor() as cursor:
        cursor.execute("SELECT * FROM samples.nyctaxi.trips LIMIT 10")
        result = cursor.fetchall()
```

Statement-level tags:

Python

```
from databricks import sql
import os

with sql.connect(
    server_hostname = os.getenv("DATABRICKS_SERVER_HOSTNAME"),  Ask Genie
    http_path       = os.getenv("DATABRICKS_HTTP_PATH"),
```

```

    access_token    = os.getenv("DATABRICKS_TOKEN"),
  ) as connection:
    with connection.cursor() as cursor:
        cursor.execute(
            "SELECT * FROM samples.nyctaxi.trips LIMIT 10",
            query_tags={"team": "engineering", "dashboard": "abc123"}
        )
    result = cursor.fetchall()

```

Node.js Connector

Minimum version: v1.12.0

JavaScript

```

const { DBSQLClient } = require('@databricks/sql');
const client = new DBSQLClient();

client
  .connect({
    host: process.env.DATABRICKS_SERVER_HOSTNAME,
    path: process.env.DATABRICKS_HTTP_PATH,
    token: process.env.DATABRICKS_TOKEN,
  })
  .then(async (client) => {
    const session = await client.openSession({
      configuration: {
        query_tags: 'team:engineering,env:prod',
      },
    });

    const queryOperation = await session.executeStatement('SELECT * FROM
samples.nyctaxi.trips LIMIT 10');
    const result = await queryOperation.fetchAll();

    await queryOperation.close();
    await session.close();
    await client.close();
  })
  .catch((error) => {
    console.error(error);
  });

```

Go Connector

 Ask Genie

Minimum version: v1.9.0

DSN connection string:

Go

```
package main

import (
    "database/sql"
    "fmt"
    _ "github.com/databricks/databricks-sql-go"
)

func main() {
    dsn :=
        "token:dapi1234@myworkspace.cloud.databricks.com:443/sql/1.0/endpoints/abc12
        query_tags=team:engineering,env:prod"

    db, err := sql.Open("databricks", dsn)
    if err != nil {
        panic(err)
    }
    defer db.Close()

    rows, err := db.Query("SELECT * FROM samples.nyctaxi.trips LIMIT 10")
    if err != nil {
        panic(err)
    }
    defer rows.Close()
}
```

NewConnector with WithSessionParams:

Go

```
package main

import (
    "database/sql"
    "os"
    dbsql "github.com/databricks/databricks-sql-go"
)

func main() {
```

 Ask Genie

```

connector, err := dbsql.NewConnector(
    dbsql.WithAccessToken(os.Getenv("DATABRICKS_ACCESS_TOKEN")),
    dbsql.WithServerHostname(os.Getenv("DATABRICKS_HOST")),
    dbsql.WithPort(443),
    dbsql.WithHTTPPath(os.Getenv("DATABRICKS_HTTP_PATH")),
    dbsql.WithSessionParams(map[string]string{
        "query_tags": "team:engineering,env:prod",
    }),
)
if err != nil {
    panic(err)
}

db := sql.OpenDB(connector)
defer db.Close()

rows, err := db.Query("SELECT * FROM samples.nyctaxi.trips LIMIT 10")
if err != nil {
    panic(err)
}
defer rows.Close()
}

```

JDBC driver (OSS)

Minimum version: v3.0.3

Connection URL:

Java

```

String url =
    "jdbc:databricks://myworkspace.cloud.databricks.com:443/default;" +
        "httpPath=/sql/1.0/endpoints/abc123;" +
        "query_tags=team:engineering,env:prod;" +
        "AuthMech=3;UID=token;PWD=dapi1234";

Connection conn = DriverManager.getConnection(url);
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM samples.nyctaxi.trips
LIMIT 10");

```

Properties object:

Java

```
String url =
"jdbc:databricks://myworkspace.cloud.databricks.com:443/default";
Properties properties = new Properties();
properties.put("httpPath", "/sql/1.0/endpoints/abc123");
properties.put("query_tags", "team:engineering,env:prod");
properties.put("AuthMech", "3");
properties.put("UID", "token");
properties.put("PWD", "dapi1234");

Connection conn = DriverManager.getConnection(url, properties);
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM samples.nyctaxi.trips
LIMIT 10");
```

JDBC driver (Simba)

Connection URL:

Java

```
String url = "jdbc:databricks://myworkspace.cloud.databricks.com:443;" +
"httpPath=/sql/1.0/endpoints/abc123;" +
"sdp_query_tags=team:engineering,env:prod;" +
"AuthMech=3;UID=token;PWD=dapi1234";

Connection conn = DriverManager.getConnection(url);
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM samples.nyctaxi.trips
LIMIT 10");
```

Properties object:

Java

```
String url = "jdbc:databricks://myworkspace.cloud.databricks.com:443";
Properties properties = new Properties();
properties.put("httpPath", "/sql/1.0/endpoints/abc123");
properties.put("sdp_query_tags", "team:engineering,env:prod");
properties.put("AuthMech", "3");
properties.put("UID", "token");
```

 Ask Genie

```
properties.put("PWD", "dapi1234");

Connection conn = DriverManager.getConnection(url, properties);
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM samples.nyctaxi.trips
LIMIT 10");
```

ODBC driver

Include the `ssp_query_tags` parameter in your ODBC connection configuration. You must also set `ApplySSPWithQueries=0` in your ODBC driver connection configuration.

View query tags

Query the `system.query.history` table to view query tags. You can group and filter by specific keys or key-value pairs.

SQL

```
SELECT statement_id, query_tags, executed_by, start_time
FROM system.query.history
WHERE MAP_CONTAINS_KEY(query_tags, 'team')
      AND query_tags['team'] = 'engineering'
ORDER BY start_time DESC
LIMIT 100;
```

NOTE

Key-only tags appear with a `null` value. To filter for them:

SQL

```
WHERE MAP_CONTAINS_KEY(query_tags, 'key') AND query_tags['key'] IS
NULL
```

For more information, see [Query history system table reference](#).

Limitations

- Query tags are limited to 10 KB per session. If the total size exceeds this limit, incoming query tags are dropped and a `tags_dropped: true` sentinel tag is added.
- Each query supports a maximum of 20 user-specified tags. When using non-SQL interfaces (session configuration parameters), additional tags are discarded and a `tag_truncated: true` sentinel tag is added.
- Tag keys and values must not exceed 128 characters. When using non-SQL interfaces (session configuration parameters), invalid tags are discarded and `tag_invalid` is added.
- Tag keys must not contain the characters `,`, `:`, `-`, `/`, `=`, or `.`. When using non-SQL interfaces (session configuration parameters), invalid tags are discarded and `tag_invalid` is added. SQL statements with invalid tag keys fail with an error at execution time.
- Query tags are supported for Databricks SQL workloads only. The `query_tags` column is not populated for other compute types.
- Keys starting with `@@` are reserved for internal use. Avoid this prefix to prevent conflicts.

Is this page

as this page helpful?

☐ Yes

☐ No

Last updated on **Apr 29, 2026**

What are SQL warehouse sessions?

SQL warehouse sessions allow you to define variables, create temporary views, and maintain state changes across multiple query runs. With sessions, you can build SQL logic iteratively without needing to run all statements at once.

You can use sessions in the following contexts when attached to a SQL warehouse:

- [SQL editor](#) queries
- [Notebooks](#)
- [Workspace](#) `.sql` files

Example use case

Sessions enable you to run statements individually while maintaining shared state. For example, you can declare a variable, create a temporary view that uses that variable, and then query the view separately.

The following example demonstrates this workflow:

SQL

```
-- Statement 1: Declare a variable
DECLARE VARIABLE row_limit = 10;

-- Statement 2: Create a temporary view using the variable
CREATE OR REPLACE TEMPORARY VIEW filtered_data AS
SELECT
  *
FROM
  main.default.sales_data
LIMIT row_limit;
```



Ask Genie

```
-- Statement 3: Query the temporary view
SELECT
  *
FROM
  filtered_data;
```

With sessions, you can run each statement individually. The variable and temporary view remain available, allowing you to iterate on your queries without re-running all previous statements.

How sessions work

A session is created when a user runs a query on a SQL warehouse. Sessions remain active as long as a command runs at least once every eight hours. Sessions persist even if the warehouse stops or restarts.

A session expires after eight hours of inactivity.

Shared session state

All commands run within the same session share the following:

- Variables declared with `DECLARE VARIABLE`
- Temporary views created with `CREATE TEMPORARY VIEW`
- Temporary tables created with `CREATE TEMPORARY TABLE`. See [Temporary tables](#)
- Environment settings such as the current catalog and schema
- Query tags set with `SET QUERY_TAGS` or session configuration parameters. See [Query tags](#)

Session scope and collaboration

Sessions are tied to both the query object (query, notebook, or file) and the specific SQL warehouse it is attached to. All users who share access to that object and warehouse connection also share the same session.

For example:

- User A defines a temporary view in Query 1 while connected to Warehouse X.
- User B can run a SQL command in Query 1 on Warehouse X that references that same temporary view.
- If Query 1 is reattached to a different warehouse, a new session is created with its own isolated state.

This collaborative behavior allows teams to work together on shared query development, but it also means that changes made by one user affect all other users sharing the same session.

Next steps

- [SQL editor](#)
- [Connect a notebook to a SQL warehouse](#)
- [Workspace files](#)

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **Apr 29, 2026**

Query caching

Caching is an essential technique for improving the performance of data warehouse systems by avoiding the need to recompute or fetch the same data multiple times. In Databricks SQL, caching can significantly speed up query execution and minimize warehouse usage, resulting in lower costs and more efficient resource utilization. Each caching layer improves query performance, minimizes cluster usage, and optimizes resource utilization for a seamless data warehouse experience.

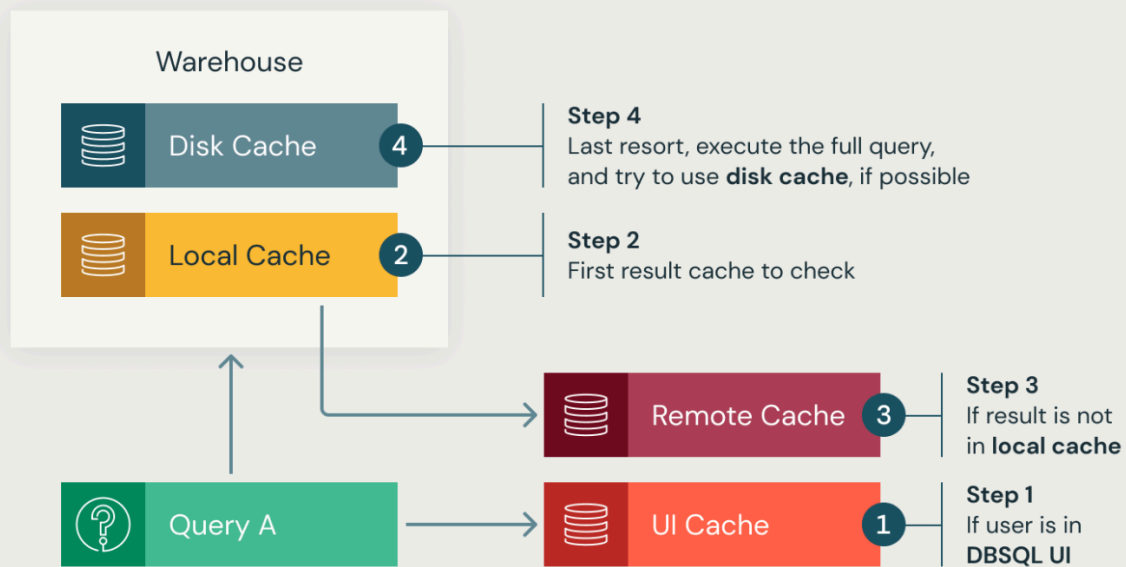
Caching provides numerous advantages in data warehouses, including:

- **Speed:** By storing query results or frequently accessed data in memory or other fast storage mediums, caching can dramatically reduce query execution times. This storage is particularly beneficial for repetitive queries, as the system can quickly retrieve the cached results instead of recomputing them.
- **Reduced cluster usage:** Caching minimizes the need for additional compute resources by reusing previously computed results. This reduces the overall warehouse uptime and the demand for additional compute clusters, leading to cost savings and better resource allocation.

Types of query caches in Databricks SQL

Databricks SQL performs several types of query caching.

Types of Caches



- **Databricks SQL UI cache:** Per user caching of all query and dashboard results in the Databricks SQL UI. When users first open a dashboard or SQL query, the Databricks SQL UI cache displays the most recent query result, including the results from scheduled executions.

The Databricks SQL UI cache has at most a 7-day life cycle. The cache is located within your Databricks filesystem in your account. You can delete query results by re-running the query that you no longer want to be stored. Once re-run, the old query results are removed from cache. Additionally, the cache is invalidated once the underlying tables have been updated.

- **Result cache:** Per cluster caching of query results for all queries through SQL warehouses. Result caching includes both local and remote result caches, which work together to improve query performance by storing query results in memory or remote storage mediums.
 - **Local cache:** The local cache is an in-memory cache that stores query results for the cluster's lifetime or until the cache is full, whichever comes first. This cache is useful for speeding up repetitive queries, eliminating the need to recompute the same results. However, once the cluster is stopped or restarted, the cache is cleaned and all query results are removed. 🧙 Ask Genie

- **Remote result cache:** The remote result cache is a serverless-only cache system that retains query results by persisting them as workspace system data. As a result, this cache is not invalidated by the stopping or restarting of a SQL warehouse. Remote result cache addresses a common pain point in caching query results in-memory, which only remains available as long as the compute resources are running. The remote cache is a persistent shared cache across all warehouses in a Databricks workspace.

Accessing the remote result cache requires a running warehouse. When processing a query, a cluster first looks in its local cache and then looks in the remote result cache if necessary. Only if the query result isn't cached in either cache is the query executed. Both the local and the remote caches have a life cycle of 24 hours, which starts at cache entry. The remote result cache persists through the stopping or restarting of a SQL warehouse. Both caches are invalidated when the underlying tables are updated.

Remote result cache is available for queries using ODBC / JDBC clients and SQL Statement API.

To disable query result caching, you can run `SET use_cached_result = false` in the SQL editor.

❗ IMPORTANT

You should use this option only in testing or benchmarking.

- **Disk cache:** Local SSD caching for data read from data storage for queries through SQL warehouses. The disk cache is designed to enhance query performance by storing data on disk, allowing for accelerated data reads. Data is automatically cached when files are fetched, utilizing a fast intermediate format. By storing copies of the files on the local storage attached to compute nodes, the disk cache ensures the data is located closer to the workers, resulting in improved query performance. See [Optimize performance with caching on Databricks](#).

In addition to its primary function, the disk cache automatically detects changes to the underlying data files. When it detects changes, the cache is invalidated. The disk cache shares the same lifecycle characteristics as the local result cache. This means that when the cluster is stopped or restarted, the cache is cleaned and needs to be repopulated.

The query results caching and [disk cache](#) affect queries in the Databricks SQL [UI](#) and [BI](#) and other external clients.

Is this page

as this page helpful?

☐ Yes	☐ No
Send feedback	

Last updated on **Apr 29, 2026**

Query filters

A query filter lets you interactively reduce the amount of data shown in a visualization. Query filters are similar to query parameter but with a few key differences. A query filter limits data *after* the query has been executed. This makes filters ideal for smaller datasets and environments where query executions are time-consuming, rate-limited, or costly.

The following describes some benefits of Databricks SQL.

- While previous query filters operated client-side only, these updated filters work dynamically on either client- or server-side to optimize performance.
- Simplified UI experience: click the **+Add filter** button and select a column from a dropdown to add a filter. You don't need to author, permission, and refresh a separate query in order to filter on the distinct values of a column.
- Enable “highlight relevant values” to see which selections within a filter will return results given other filter selections. For example, consider a user who has both a “State” and “City” filter. If a user chooses to highlight relevant values, selecting “California” in the state filter will highlight only the cities in California in the “City” filter. Non-highlighted options are put under a “Filtered out” menu option in the dropdown.
- Text Input filters: filters column results based on text input searches. There are three modes the search can find matches with: exact match, contains, and starts with.
- Quick date selectors enable you to filter on predefined date ranges such as last week, last month, last year, and more.
- You can set default date ranges when creating date filters.

Description of functionality

After running a query, in the **Results** panel, click **+** and then select **Filter**



Ask Genie

The **+Add filter** button opens a popup menu where you can apply the following filters and settings.

- Column: the column on which to apply the filter.
 - Strings, numbers, and dates are currently supported.
 - If the selected column contains dates, users can choose to specify a time binning by date, month, or year.
- Type: the type of filter to apply
 - Single Select: filter to one field value only
 - Multi Select: filter to multiple field values
 - Text Input: enter a string to search for matching values in a particular column. Supports "Contains," "Exact Match," and "Starts With"
 - Date / time picker and ranger picker: ability to select a date or date range to filter on date data types
- Sort Order: The order in which values are sorted for the filter dropdown. You can choose between "Ascending" or "Descending."
- Highlight relevant values (y/n): Enable this to easily see which selections within a filter will return results given other filter selections. For example, consider a user who has both a "State" and "City" filter. If a user chooses to highlight relevant values, selecting "California" in the state filter will highlight the set of options available in the "City" filter to only show cities in California, while non-highlighted options will be put under a "Filtered out" menu option in the dropdown. Note that this requires running a query each time a filter is updated.
- Default value: When a "date" type column is selected and a time-binned value is chosen (days, months, or years), users can also choose to set a default date range for the filter. The default filter is automatically applied whenever the query is refreshed.

Limitations

- It is important to note that query filters are applied over the entirety of the dataset. However, the dropdown selector for query filters is limited to 64k unique values. If a user wishes to filter in situations where there are more than 64k unique filter values, it is recommended to use a **Text** parameter instead.

- Filters can only be applied to columns returned by a query, not all columns of a referenced table.
- Filters display the distinct list of options from the designated column in returned results. If the results are limited (i.e. query run with **Limit 1000**), then a filter will only display unique values from within those 1000 results.
- While filters applied to a query will optimize to run on either the client or server side for better performance, filters applied to a dashboard will always run on the server side.

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **Apr 29, 2026**

Use named parameter markers

Named parameter markers let you insert variable values into SQL queries at runtime. Instead of hard-coding specific values, you define typed placeholders that users fill in when the query runs. This improves query reuse, prevents SQL injection, and makes it easier to build flexible, interactive queries.

Named parameter markers work in the following Databricks surfaces:

- SQL editor (new and legacy)
- Notebooks
- AI/BI dashboard dataset editor
- Genie Spaces

Add a named parameter marker

Insert a parameter by typing a colon followed by a parameter name, such as `:parameter_name`. When you add a named parameter marker to a query, a widget appears where you can set the parameter type and value. See [Work with parameter widgets](#).

This example converts a hard-coded query to use a named parameter.

Starting query:

SQL

```
SELECT
  trip_distance,
  fare_amount
FROM
  samples.nyctaxi.trips
```



 Ask Genie

WHERE

```
fare_amount < 5
```

1. Delete `5` from the `WHERE` clause.
2. Type `:fare_parameter` in its place. The last line should read `fare_amount < :fare_parameter`.
3. Click the gear icon near the parameter widget.
4. Set the **Type** to **Decimal**.
5. Enter a value in the parameter widget and click **Apply changes**.
6. Click **Save**.

Parameter types

Set the parameter type in the parameter settings panel. The type determines how Databricks interprets and handles the value at runtime.

Type	Description
String	Free-form text. Backslash, single, and double quotation marks are escaped automatically. Databricks adds quotation marks around the value.
Integer	Whole number value.
Decimal	Numeric value that supports fractional values.
Date	Date value. Uses a calendar picker and defaults to the current date.
Timestamp	Date and time value. Uses a calendar picker and defaults to the current date and time.

Named parameter syntax examples

The following examples show common patterns for named parameter markers. Ask Genie

Insert a date

SQL

```
SELECT
  o_orderdate AS Date,
  o_orderpriority AS Priority,
  sum(o_totalprice) AS `Total Price`
FROM
  samples.tpch.orders
WHERE
  o_orderdate > :date_param
GROUP BY 1, 2
```

Insert a number

SQL

```
SELECT
  o_orderdate AS Date,
  o_orderpriority AS Priority,
  o_totalprice AS Price
FROM
  samples.tpch.orders
WHERE
  o_totalprice > :num_param
```

Insert a field name

Use the `IDENTIFIER` function to pass a column name as a parameter. The parameter value should be a column name from the table used in the query.

SQL

```
SELECT * FROM samples.tpch.orders
WHERE IDENTIFIER(:field_param) < 10000
```

Insert database objects

Use the `IDENTIFIER` function with multiple parameters to specify a catalog, schema, and table at runtime.

SQL

```
SELECT *  
FROM IDENTIFIER(:catalog || '.' || :schema || '.' || :table)
```

See [IDENTIFIER clause](#).

Concatenate multiple parameters

Use `format_string` to combine parameters into a single formatted string. See [format_string function](#).

SQL

```
SELECT o_orderkey, o_clerk  
FROM samples.tpch.orders  
WHERE o_clerk LIKE format_string('%s%s', :title, :emp_number)
```

Work with JSON strings

Use the `from_json` function to extract a value from a JSON string using a parameter as the key. Substituting `a` as the value for `:param` returns `1`.

SQL

```
SELECT from_json('{\"a\": 1}', 'map<string, int>') [:param]
```

Create an interval

Use `CAST` to convert a parameter value to an `INTERVAL` type for time-based calculations. See [Interval type](#).

SQL

```
SELECT CAST(:param AS INTERVAL MINUTE)
```

Add a date range using `.min` and `.max`

Date and Timestamp parameters support a range widget. Use `.min` and `.max` to access the start and end of the range.

SQL

```
SELECT * FROM samples.nyctaxi.trips
WHERE tpep_pickup_datetime
BETWEEN :date_range.min AND :date_range.max
```

Set the parameter type to `Date` or `Timestamp` and the widget type to **Range**.

Add a date range using two parameters

SQL

```
SELECT * FROM samples.nyctaxi.trips
WHERE tpep_pickup_datetime
BETWEEN CAST(:date_range_min AS TIMESTAMP) AND CAST(:date_range_max AS
TIMESTAMP)
```

Parameterize rollup granularity

Use `DATE_TRUNC` to aggregate results at a user-selected level of granularity. Pass `DAY`, `MONTH`, or `YEAR` as the parameter value.

SQL

```
SELECT
  DATE_TRUNC(:date_granularity, tpep_pickup_datetime) AS date_rollup,
  COUNT(*) AS total_trips
```

 Ask Genie

```
FROM samples.nyctaxi.trips
GROUP BY date_rollup
```

Pass multiple values as a string

Use `ARRAY_CONTAINS`, `SPLIT`, and `TRANSFORM` to filter on a comma-separated list of values passed as a single string parameter. `SPLIT` parses the comma-separated string into an array. `TRANSFORM` trims whitespace from each element. `ARRAY_CONTAINS` checks whether the table value appears in the resulting array.

SQL

```
SELECT * FROM samples.nyctaxi.trips WHERE
  array_contains(
    TRANSFORM(SPLIT(:list_parameter, ','), s -> TRIM(s)),
    CAST(dropoff_zip AS STRING)
  )
```

NOTE

This example works for string values. To use other data types, wrap the `TRANSFORM` operation with a `CAST` to convert the elements to the desired type.

Syntax migration reference

Use this table when converting queries from mustache syntax to named parameter markers. See [Mustache parameter syntax](#) for more information about the legacy syntax.

Use case	Mustache syntax	Named parameter syntax
Filter by date	<code>WHERE date_field < '{{date_param}}'</code>	<code>WHERE date_field < :date_param</code>
Filter by number	<code>WHERE price < {{max_price}}</code>	<code>WHERE price < :max_price</code>  Ask Genie

Use case	Mustache syntax	Named parameter syntax
Compare strings	<code>WHERE region = '{{region_param}}'</code>	<code>WHERE region = :region_param</code>
Specify a table	<code>SELECT * FROM {{table_name}}</code>	<code>SELECT * FROM IDENTIFIER(:table)</code> — use the full three-level namespace
Specify catalog, schema, and table	<code>SELECT * FROM {{catalog}}. {{schema}}.{{table}}</code>	<code>SELECT * FROM IDENTIFIER(:catalog '.' :schema '.' :table)</code>
Format a string from multiple parameters	<code>"({{area_code}}) {{phone_number}}"</code>	<code>format_string("(%d) %d", :area_code, :phone_number)</code>
Create an interval	<code>SELECT INTERVAL {{p}} MINUTE</code>	<code>SELECT CAST(format_string("INTERVAL '%s' MINUTE", :param) AS INTERVAL MINUTE)</code>

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **Apr 29, 2026**

Query snippets

It's often easier to duplicate prior work and modify than to write something from scratch. This is particularly true for common `JOIN` statements or complex `CASE` expressions. As your list of queries grows, it can be difficult to remember which queries contain the statement you need. You can also create a query snippet that contains an insertion point with placeholder text that a user can replace at runtime.

Query snippets are segments of queries that you can share and trigger using autocomplete. Use query snippets for:

- Frequent `JOIN` statements
- Complicated clauses like `WITH` or `CASE`.
- Conditional formatting

Here are examples of snippets:

```
--Simple snippet
WHERE fare_amount > 100

--Snippet with an insertion point for a value to be provided at runtime
WHERE fare_amount > ${1:value}

--Snippet with an insertion point for a value to be provided at runtime
and containing a default value
WHERE fare_amount > ${1:100}

--Snippet with multiple insertion points
WHERE fare_amount > ${2:min_value} AND fare_amount < ${1:max_value} AND
trip_distance < ${0:max_distance}
```

Create query snippets

You can create a query snippet from the SQL, notebook, or file editors. Follow these steps:

1. In the [new SQL editor](#), click the  kebab menu, then click **View**.

In the notebook or file editors, click **View**.

2. Click **Query snippets**.

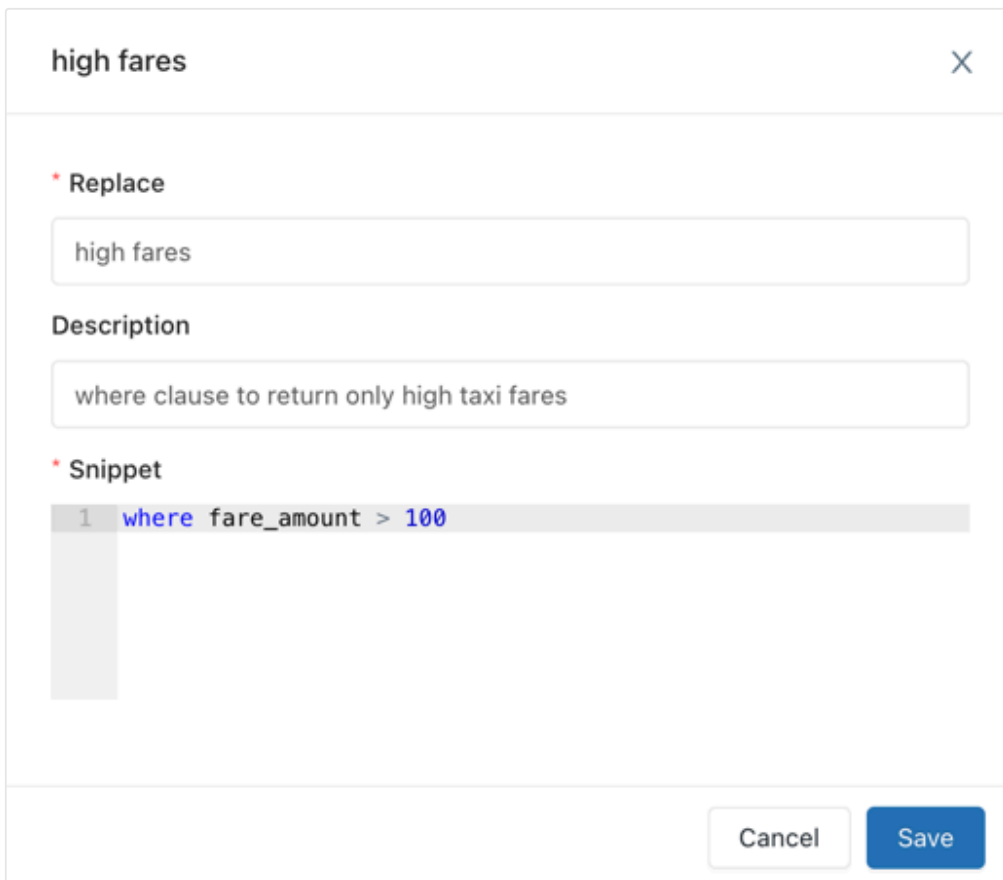
3. Click **Create query snippet**.

4. In the **Replace** field, enter the snippet name. You will use this name when writing a query that uses the snippet.

5. (Optional) Enter a description.

6. In the **Snippet** field, enter the snippet.

7. Click **Create**.



The screenshot shows a modal dialog titled "high fares" with a close button (X) in the top right corner. The dialog contains three main sections:

- * Replace**: A text input field containing the text "high fares".
- Description**: A text input field containing the text "where clause to return only high taxi fares".
- * Snippet**: A code editor area with a line of SQL code: `1 where fare_amount > 100`. The code is highlighted in blue.

At the bottom right of the dialog, there are two buttons: "Cancel" and "Save".

Use a query snippet

Query snippets can be used any SQL query. You can access query snippets in the SQL editor, notebook SQL cells, SQL files, and AI/BI dashboard queries.

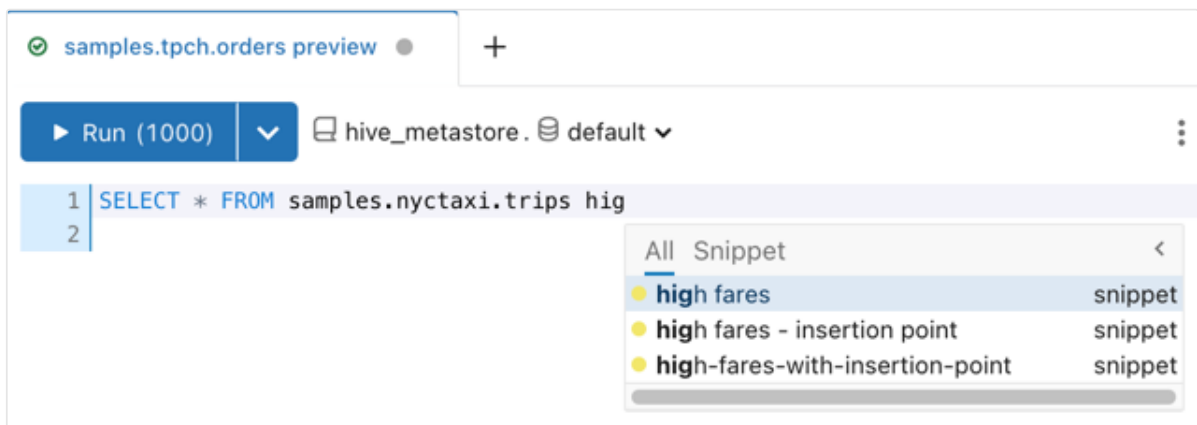
Here's an example of a simple query with which you can use these query snippets:

```
--Simple query
SELECT * FROM samples.nyctaxi.trips
```

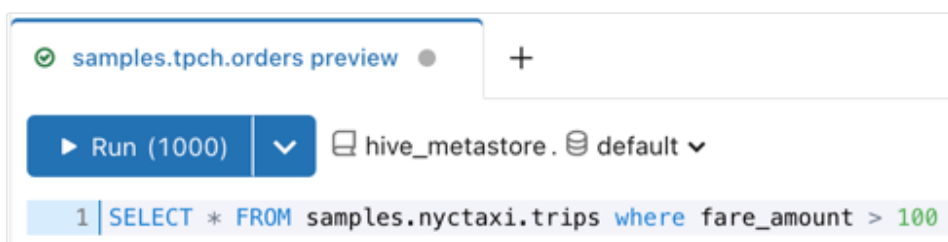
Use the following steps to use a query snippet with this query:

1. Open **SQL Editor**.
2. Type your query in the SQL editor query pane.
3. Start typing the name of your query snippet, then select it from the autocomplete window.

In the SQL editor, you can also manually open the window with **Control+Space** and select a snippet.



4. Execute the query with the **WHERE** clause from the query snippet.



Working with insertion points in query snippets

You designate insertion points by wrapping an integer tab order with a single dollar sign and curly braces `${}`. A text placeholder preceded by a colon `:` is optional but useful for users unfamiliar with your snippet. In the query snippets with insertion points that you created previously, `${1:value}` is an insertion point with a placeholder, and `${1:100}` is an insertion point with a default value for the placeholder that you can override at runtime. When Databricks SQL renders the snippet, the dollar sign `$` and curly braces `{}` are stripped away, and the word `value` or the default of `100` is highlighted for replacement.

When there are multiple insertion points, the text insertion caret jumps to the first insertion point to prompt for the desired value. When you press `Tab`, the caret jumps to the next insertion point for the next value. When you press `Tab` again, the caret will jump to the next insertion point line until it reaches the final insertion point.

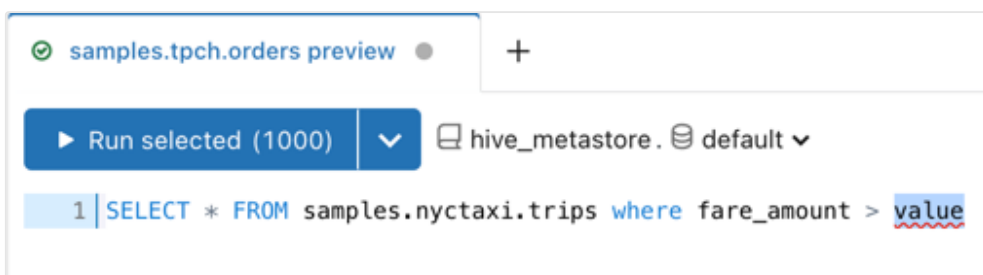
NOTE

An insertion point of zero `${0}` is always the last point in the tab order.

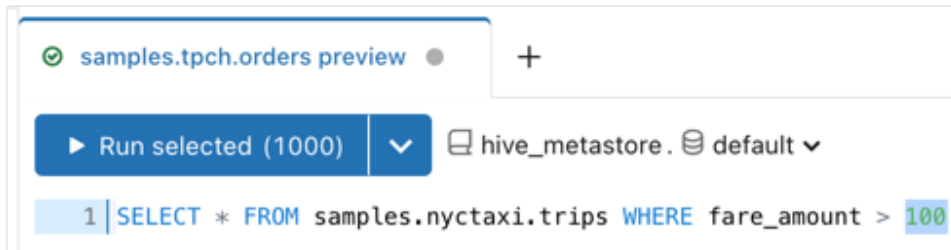
Use the following steps to use the insertion point query snippets with the query:

1. Open **SQL Editor**.
2. Type your query in the SQL editor query pane.
3. Type the first three letters of the name of your query snippet and then select a query snippet with the insertion point without a default value.

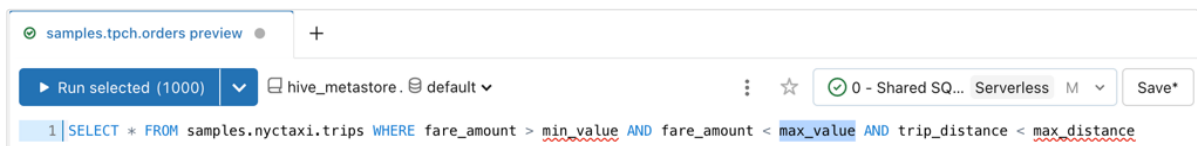
The query snippet is added to the query, and the text insertion caret jumps to the insertion point.



4. Enter a value for the `WHERE` clause, such as `200`.
5. Optionally, run the query with the `WHERE` clause from the query snippet.
6. Repeat the previous steps, but select the query snippet with the insertion point using a default value.



7. Repeat the previous steps, but select the query snippet with multiple insertion points.



8. Enter a value for the first insertion point, tab to the next insertion point, and enter a value, and then tab to the final insertion point and enter a value.

Is this page

as this page helpful?

☐ Yes

☐ No

Send feedback

Last updated on **Apr 29, 2026**

Query profile

You can use a query profile to visualize the details of a query execution. The query profile helps you troubleshoot performance bottlenecks during the query's execution. For example:

- You can visualize each query operator and related metrics, such as the time spent, number of rows processed, rows processed, and memory consumption.
- You can identify the slowest part of a query execution at a glance and assess the impacts of modifications to the query.
- You can discover and fix common mistakes in SQL statements, such as exploding joins or full table scans.

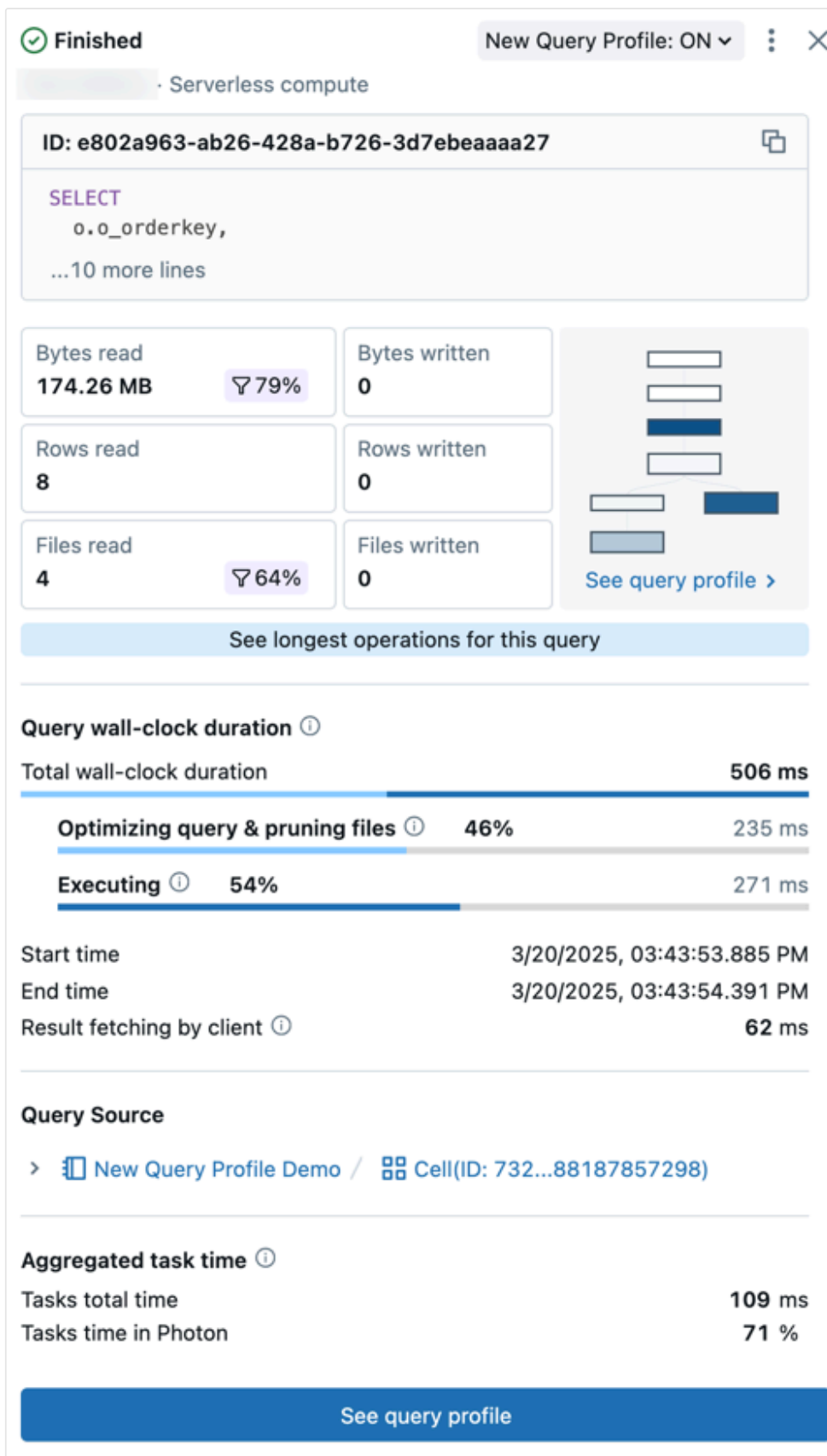
Requirements

To view a query profile, you must either be the owner of the query or you must have at least CAN MONITOR [permission](#) on the SQL warehouse that executed the query.

View a query profile

You can view the query profile from the query history using the following steps:

1. Click  **Query History** in the sidebar.
2. Click the name of a query. A query details panel appears on the right side of the screen.

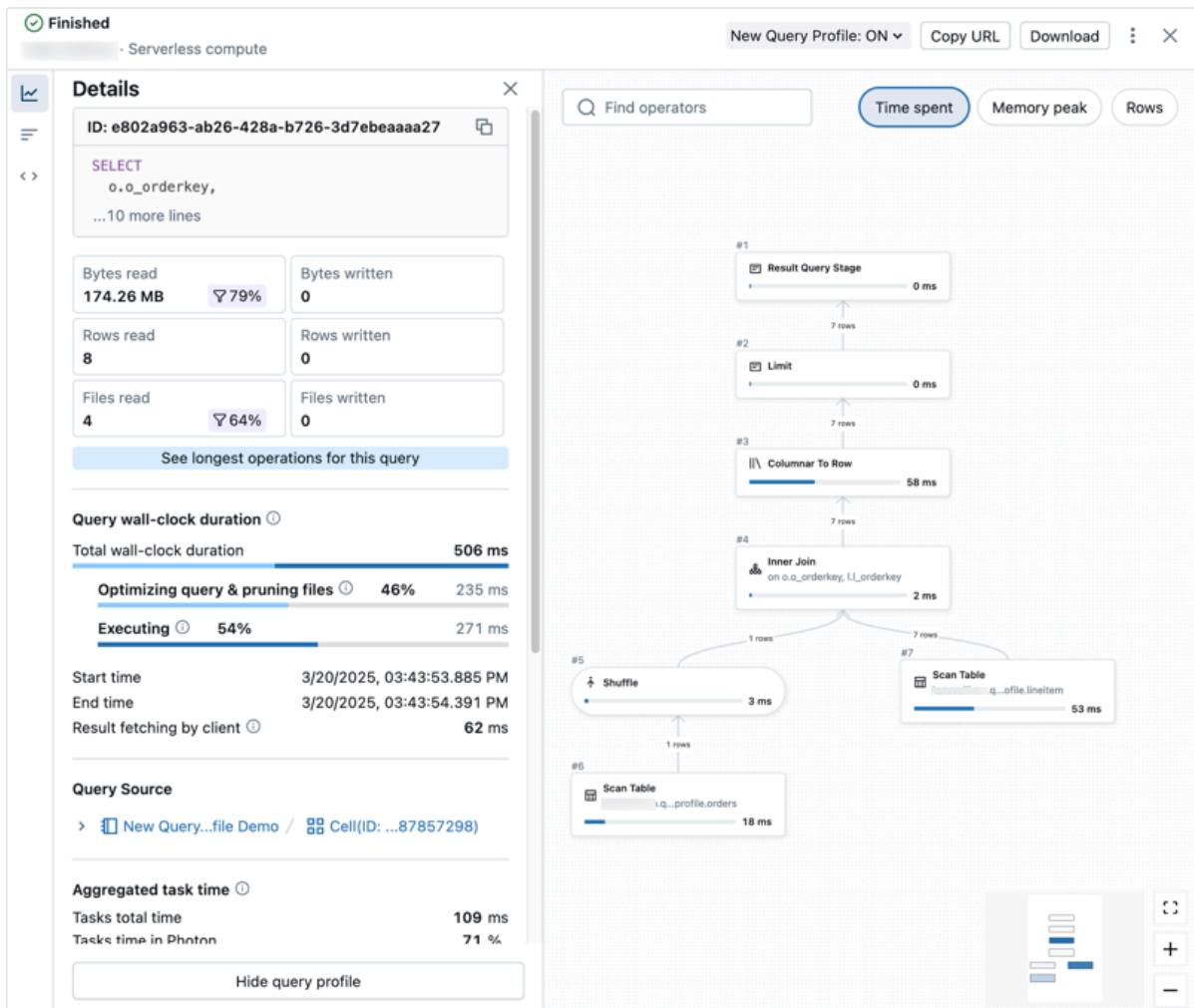


The query summary includes:

- **Query status:** The query is tagged with its current status: **Queued**, **Running**, **Finished**, **Failed**, or **Cancelled**.

- **User and compute details:** See the username, compute type, and runtime details for this query run.
- **ID:** This is the universally unique identifier (UUID) associated with the given query execution.
- **Query statement:** This section includes the complete query statement. If the query is too long to be shown in the preview, click ... **more lines** to view the full text.
- **Query metrics:** Popular metrics for query analysis are shown under the query text. The filter icons that appear with some metrics indicate the percent of data pruned during scanning.
- **See query profile:** A preview of the query profile directed acyclic graph (DAG) is shown in this summary. This can be helpful for quickly estimating query complexity and execution flow. Click **See query profile** to open the detailed DAG.
- **See longest operators for this query:** Click this button to open the **Top operators** panel. This panel shows the longest running operators in the query.
- **Query wall-clock duration:** The total elapsed time between the start of scheduling and the end of the query execution is provided as a summary. A detailed breakdown of scheduling, query optimization and file pruning, and execution time appears below the summary.
- **Query source:** Click on the name of the listed object to go to the query source.
- **Aggregated task time:** View the combined time it took to execute the query across all cores of all nodes. It can be significantly longer than the wall-clock duration if multiple tasks are executed in parallel. It can be shorter than the wall-clock duration if tasks waited for available nodes.
- **Input/Output (IO):** View details about the data read and written during the query execution.

3. Click **See query profile**. A **Details** panel opens on the right side of the screen.



NOTE

If **Query profile is not available** is displayed, no profile is available for this query. A query profile is not available for queries that run from the [query cache](#). To circumvent the query cache, make a trivial change to the query, such as changing or removing the `LIMIT`.

View query profile details

The detailed query profile includes summary metrics on the left side of the panel and a graph view of operators on the right.

Explore query metrics

The left side of the query profile has the following tabs:

1.  **Details:** Opens the **Details** panel which shows [query summary metrics](#).
2.  **Top operators:** Opens the **Top operators** panel which shows the most expensive operators used in your query. This can be useful for identifying optimization opportunities.
3.  **Query text:** Opens the **Query text** panel which shows the full text of the query.

 NOTE

Some non-Photon operations are executed as a group and share common metrics. In this case, all operations have the same value as the parent operator for a given metric.

Explore the DAG

The right half of the query profile shows the directed acyclic graph (DAG) of the query. The graph view shows metrics such as **Time spent**, **Memory peak**, and **Rows**. Click on each metric to change the reporting metric shown.

You can interact with the DAG in the following ways:

- Use the search bar to highlight different operators or columns.
- Zoom in or out focus on different parts of the DAG.
- Click on operators to show detailed metrics and descriptions. A panel on the right side of the graph shows operation details.

For Databricks SQL queries, you can also view the query profile in the Spark UI. Click the  kebab menu near the top of the page, then click **Open in Spark UI**.

By default, metrics for some operations are hidden. These operations are unlikely to be the cause of performance bottlenecks. To see information for all operations, and to see additional metrics, click  at the top of the page, then click **Enable verbose mode**.

Common operations

The most common operations are:

- **Scan:** Data was read from a datasource and output as rows.
- **Join:** Rows from multiple relations were combined (interleaved) into a single set of rows.
- **Union:** Rows from multiple relations that use the same schema were concatenated into a single set of rows.
- **Shuffle:** Data was redistributed or repartitioned. Shuffle operations are expensive with regard to resources because they move data between executors on the cluster.
- **Hash / Sort:** Rows were grouped by a key and evaluated using an aggregate function such as `SUM`, `COUNT`, or `MAX` within each group.
- **Filter:** Input is filtered according to a criteria, such as by a `WHERE` clause, and a subset of rows is returned.

Share a query profile

To share a query profile with another user:

1. View [query history](#).
2. Click the name of the query.
3. To share the query, you have two choices:
 - If the other user has the CAN MANAGE permission on the query, you can share the URL for the query profile with them. Click **Share**. The URL is copied to your clipboard.
 - Otherwise, if the other user does not have the CAN MANAGE permission or is not a member of the workspace, you can download the query profile as a JSON object. **Download**. The JSON file is downloaded to your local system.

Import a query profile

To import the JSON for a query profile:

1. View [query history](#).
2. Click the kebab menu  on the upper right, and select **Import query profile (JSON)**.  Ask Genie

3. In the file browser, select the JSON file that was shared with you and click **Open**. The JSON file is uploaded and the query profile is displayed.

When you import a query profile, it is dynamically loaded into your browser session and does not persist in your workspace. You need to re-import it each time you want to view it.

4. To close the imported query profile, click **X** at the top of the page.

Access the query profile

You can also access the query profile in the following parts of the UI:

- **From the SQL editor:** During and after query execution, a link near the bottom of the page displays the time elapsed and number of rows returned. Click that link to open the **query details** panel. Click **See query profile**.

 3 s 136 ms | 10 rows returned

NOTE

If you have the [new SQL editor](#) enabled (Public Preview), your link appears as it does in a notebook.

- **From a notebook:** If your notebook is attached to a SQL warehouse or serverless compute, you can access the query profile using the link under the cell that contains the query. Click **See performance** to open the run history. Click a statement to open the **query details** panel.

>  See performance (2)

- **From the Lakeflow Spark Declarative Pipelines UI:** You can access the query history and profile from the **Query History** tab in the pipeline UI. See [Access query history for pipelines](#).
- **From the jobs UI:** You can access query profiles for jobs run on SQL warehouses and serverless compute. For jobs run on serverless compute, see [View query details for job runs](#) to learn how to view query details in the jobs UI.  Ask Genie

Next steps

- Learn about accessing query metrics using the [query history API](#)
- Learn more about [query history](#)

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **May 11, 2026**

Query performance insights

ⓘ PREVIEW

This feature is in [Private Preview](#). To try it, reach out to your Databricks contact.

When queries run, Databricks might return insights that identify opportunities to improve performance. This page lists the supported insights and their meaning.

For a broader overview of performance best practices, review the [Comprehensive Guide to Optimize Databricks, Spark and Delta Lake Workloads](#).

CONCURRENT_WRITE

- Concurrent writes on the table cause [conflicts](#) that are automatically resolved or fail.
- **Recommendation:** Review the [delta history](#) to identify concurrent writes and consider different scheduling to avoid conflicts.

COVERAGE_FILTER_KEYS_CLUSTERING

- The table is [clustered](#) by one or more keys that aren't used in filtering during the table scan.
- **Recommendation:** Determine which data subset you need for the desired outcome, then add filters on matching clustering keys to reduce bytes read.

COVERAGE_FILTER_KEYS_PARTITIONING

#

 Ask Genie

- The table is [partitioned](#) by one or more keys that aren't used in filtering during the table scan.
- **Recommendation:** Determine which data subset you need for the desired outcome, then add filters on matching partitioning keys to reduce bytes read.

COVERAGE_PHOTON

- [Photon](#) can't accelerate the operation, so the standard runtime engine was used.
- **Recommendation:** Review Photon limitations and consider adjusting the query to use a supported execution strategy for faster runtime.

COVERAGE_STATS_DELTA

- [Delta data skipping](#) statistics are missing or incomplete for the table scan file filters, so the query uses in-file filtering. The following statistics statuses are possible:
 - *Full*: Statistics are available for all filters.
 - *Partial*: Statistics are available on a subset of filters.
 - *Unavailable*: Statistics are not available on any filter.
 - *Unused*: Statistics could not be used on a filter that converts the data type.
- **Recommendation:** [Collect Delta statistics](#) to reduce the number of bytes read.

COVERAGE_STATS_OPTIMIZER

- [Cost-based optimizer](#) statistics are missing or incomplete, so standard heuristics were used to generate the query plan.
- **Recommendation:** [Collect statistics](#) to enable the optimizer to produce a better plan.

DATA_SKEW

- [Data is processed unevenly](#) by available computing resources.

- **Recommendation:** Review the distribution of the data, then salt keys or pre-aggregate the data.

DATA_SPILL

- [Data spill](#) to disk while executing an operator because the data size did not fit in memory.
- **Recommendation:** Increase warehouse [size](#) to increase available memory. Reduce the number of rows, number of columns or size of large column (strings, arrays, maps, structs) to reduce memory usage.

EXCESSIVE_QUEUE_TIME

- Query has been waiting in queue on [warehouse](#).
- **Recommendation:** Increase the maximum [number of clusters](#) on the warehouse to reduce queue time.

EXPLODING_JOIN

- [Join](#) is [generating significantly more rows](#) than it has read.
- **Recommendation:** Determine which result subset is required, then update the join or reduce the number of input rows from both relations.

FLOW_FULL_RECOMPUTE

- [Flow](#) has been planned to be executed as [full recompute](#).
- **Recommendation:** Rewrite the query for [incremental support](#) to reduce the number of bytes read.

IO_THROTTLING

- [Cloud storage](#) request was throttled by your cloud provider.

- **Recommendation:** Contact your administrator to increase your cloud storage request limits with your cloud provider.

REDUNDANT_AGGREGATION

- [Aggregate](#) did not change the query result.
- **Recommendation:** Remove the aggregate or apply [primary and foreign key constraints](#).

SELECTIVE_JOIN

- [Join](#) is generating significantly fewer rows than it has read.
- **Recommendation:** Determine which result subset is required, then add filters before the join to reduce the number of input rows.

WIDE_PROJECTION

- [Projecting](#) all columns on the table.
- **Recommendation:** Determine which result subset is required, then project only those columns to reduce the number of bytes read.

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **Apr 29, 2026**

Query optimization using primary key constraints

Primary key constraints, which capture relationships between fields in tables, can help users and tools understand relationships in your data. This article contains examples that show how you can use primary keys with the `RELY` option to optimize some common types of queries.

NOTE

Query optimizations associated with the `RELY` command require queries to run on Photon-enabled compute. See [What is Photon?](#). Photon runs by default on SQL warehouses and serverless compute for notebooks and workflows. To learn more about Photon, see [What is Photon?](#).

Add primary key constraints

You can add a primary key constraint in your table creation statement, as in the following example, or add a constraint to a table using the `ADD CONSTRAINT` clause.

SQL

```
CREATE TABLE customer (  
  c_customer_sk int,  
  PRIMARY KEY (c_customer_sk)  
  ...  
)
```

In this example, `c_customer_sk` is the customer ID key. The primary key constraint specifies that each customer ID value should be unique in the table. Databricks does not enforce key constraints. They can be validated through your existing [Ask Genie](#)

or ETL. See [Manage data quality with pipeline expectations](#) to learn about working expectations on streaming tables and materialized views. See [Constraints on Databricks](#) to learn about working with constraints on Delta tables.

NOTE

It is the user's responsibility to check whether a constraint is satisfied. Relying on a constraint that is not satisfied can lead to incorrect query results.

Use `RELY` to enable optimizations

When you know that a primary key constraint is valid, you can enable optimizations based on the constraint by specifying it with the `RELY` option. See [ADD CONSTRAINT clause](#) for the complete syntax.

The `RELY` option allows Databricks to exploit the constraint to rewrite queries. The following optimizations can only be performed if the `RELY` option is specified in an `ADD CONSTRAINT` clause or `ALTER TABLE` statement.

Using `ALTER TABLE`, you can modify a table's primary key to include the `RELY` option, as shown in the following example.

SQL

```
ALTER TABLE
  customer DROP PRIMARY KEY;
ALTER TABLE
  customer
ADD
  PRIMARY KEY (c_customer_sk) RELY;
```

Optimization examples

The following examples extend the previous example that creates a `customer` table where `c_customer_sk` is a verified unique identifier named as a `PRIMARY KEY` with the `RELY` option specified.

 Ask Genie

Example 1: Eliminate unnecessary aggregations

The following shows a query that applies a `DISTINCT` operation to a primary key.

SQL

```
SELECT
  DISTINCT c_customer_sk
FROM
  customer;
```

Because the `c_customer_sk` column is a verified `PRIMARY KEY` constraint, all values in the column are unique. With the `RELY` option is specified, Databricks can optimize the query by not performing the `DISTINCT` operation.

Example 2: Eliminate unnecessary joins

The following example shows a query where Databricks can eliminate an unnecessary join.

The query joins a fact table, `store_sales` with a dimension table, `customer`. It performs a left outer join, so the query result includes all records from the `store_sales` table and matched records from the `customer` table. If there is no matching record in the `customer` table, the query result shows a `NULL` value for the `c_customer_sk` column.

SQL

```
SELECT
  SUM(ss_quantity)
FROM
  store_sales ss
  LEFT JOIN customer c ON ss.customer_sk = c.c_customer_sk;
```

To understand why this join is unnecessary, consider the query statement. It requires only the `ss_quantity` column from the `store_sales` table. The `customer` table is joined on its primary key, so each row of `store_sales` matches at most one row in `customer`. Because the operation is an outer join, all records from the `store_sales` table are included.

table are preserved, so the join does not change any data from that table. The `SUM` aggregation is the same whether or not these tables are joined.

Using the primary key constraint with `RELY` gives the query optimizer the information it needs to eliminate the join. The optimized query looks more like this:

SQL

```
SELECT
  SUM(ss_quantity)
FROM
  store_sales ss
```

Next steps

See [View the Entity Relationship Diagram](#) to learn how to explore primary key and foreign key relationships in the Catalog Explorer UI.

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback